

Steering Behaviors

06

Steering behaviors bring virtual worlds to life, allowing autonomous agents to move intelligently—seeking targets, avoiding obstacles, and adapting dynamically to their surroundings, resulting in lifelike motion in games and simulations.



In open-world simulations or video games, have you ever noticed how NPCs (non-player characters) navigate the environment—avoiding obstacles, chasing the player, or patrolling an area? For example, in strategy games like **Dota 2** or **Company of Heroes**, AI-controlled entities follow paths, engage enemies, and adjust their movement dynamically during battles.



Image Credits: commons.wikimedia

Similarly, in open-world games like Minecraft, characters such as villagers, animals, and hostile creatures move naturally - wandering, avoiding obstacles, and interacting with the environment. These behaviors aren't manually scripted for each entity; instead, they are driven by special algorithms called **Steering Algorithms**, which allow agents to move intelligently and respond dynamically to their surroundings.

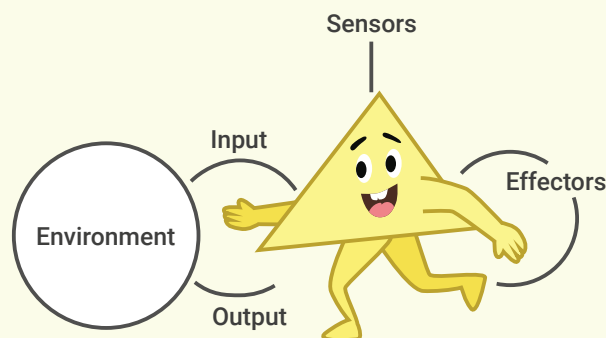


Image Credits: minecraft.com

In earlier lessons, we explored how vectors can represent position, velocity, and acceleration, allowing objects to move realistically in a virtual space. We used these concepts to create autonomous agents, applying basic physics principles to simulate motion.

Recall:

Autonomous Agents are virtual characters or objects that possess a degree of autonomy in generating their own movements and actions. Rather than relying on pre-defined programmed movements and animations, these agents use algorithms and rules to generate actions dynamically in real-time.



They have a wide range of applications across different fields, such as simulation and modelling, games, virtual environment applications, AR/VR, etc.

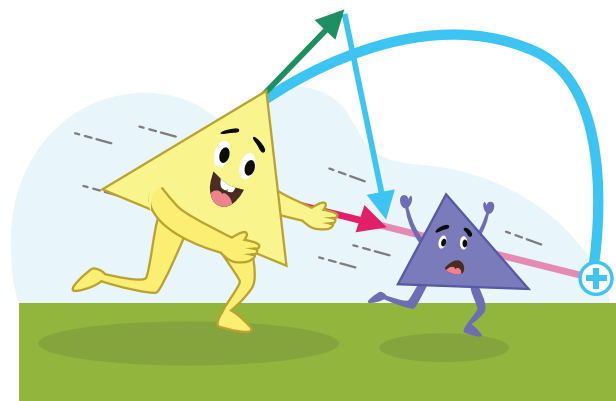
Now, we will move beyond simple movement and explore Steering Behaviors.

What Are Steering Behaviors?

Steering Behaviors allow an agent to adjust its motion dynamically, rather than simply following pre-programmed paths.

These behaviors help agents:

1. Seek a moving target.
2. Arrive smoothly at a destination.
3. Wander randomly in a controlled way.
4. Avoid obstacles and react to surroundings.
5. Pursue or evade other moving objects.



Instead of moving in a straight line towards a target, an agent considers its current velocity and direction before adjusting its motion. This approach creates a lifelike movement that feels organic, rather than robotic.

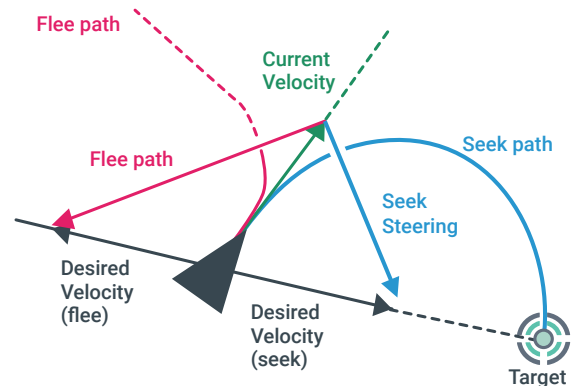
Steering Algorithms

In the late 1980s, computer scientist Craig Reynolds created steering behaviors for animated characters. These behaviors helped digital characters move in lifelike ways—like fleeing, wandering, arriving, or chasing.

In his 1999 research paper “*Steering Behaviors for Autonomous Characters*,” Reynolds called these agents or characters as ‘vehicles’ and focused on how they behave, not how they look. He explained three main steps:

- **Action Selection** – The vehicle chooses what to do based on a goal. For example, if it sees danger, it runs away. Other goals could be following a path or avoiding obstacles.
- **Steering** – After choosing an action, it calculates how to move using a steering force.
- **Movement** – This represents the actual motion, such as sliding across the screen.

The most important step is action selection. Realistic motion is created by combining several actions to make the character’s behavior appear natural and dynamic.



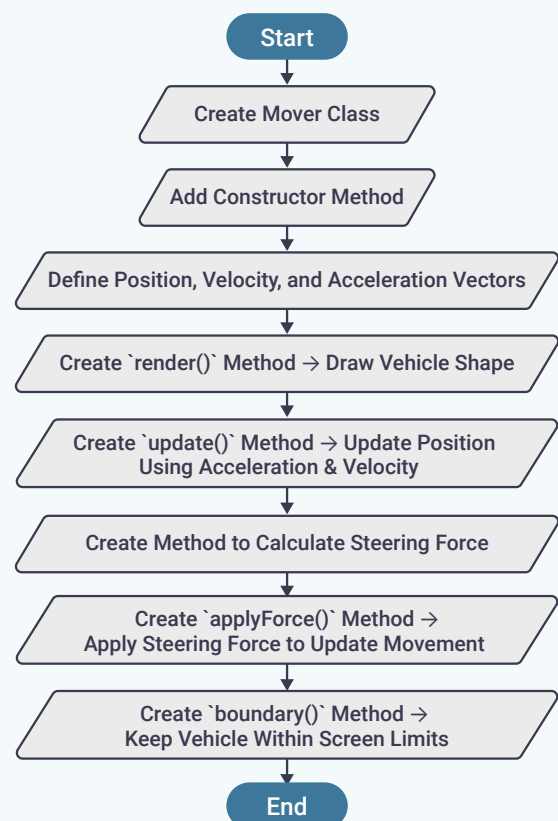
Activity

In this lesson, we will begin with a given program for an autonomous agent that will walk randomly with the implementation of the concepts of:

- Vectors for movement calculations.
- Object-Oriented Programming to structure our agent.
- Force, acceleration and laws of motion.

This will save us time by building on the groundwork covered in earlier grades, allowing us to focus on steering behaviors like seek, arrive, and avoidance.

We will begin by reviewing the provided code to reinforce earlier concepts and understand its structure. The accompanying flowchart will serve as a guide for recalling the process.



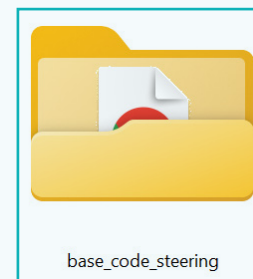
Activity

Part 1 - Initial File Setup

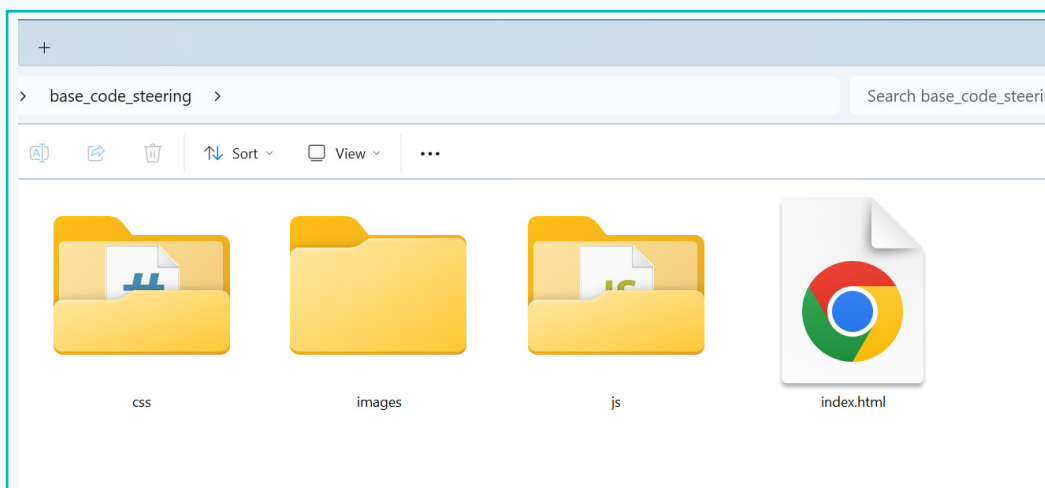
- 1 Download the project folder using the given QR code.



- 2 Rename the folder with a suitable name.



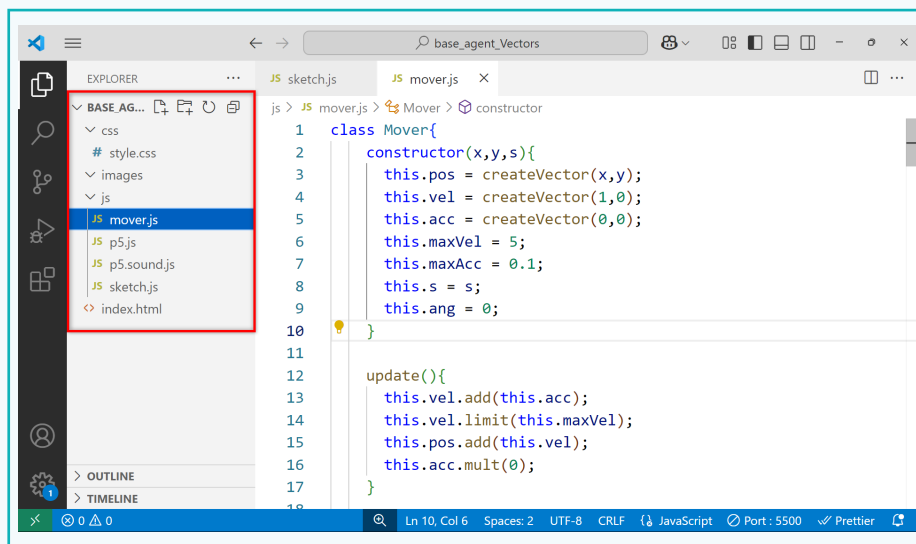
The folder contains all the necessary files required as a starting point for the activity.



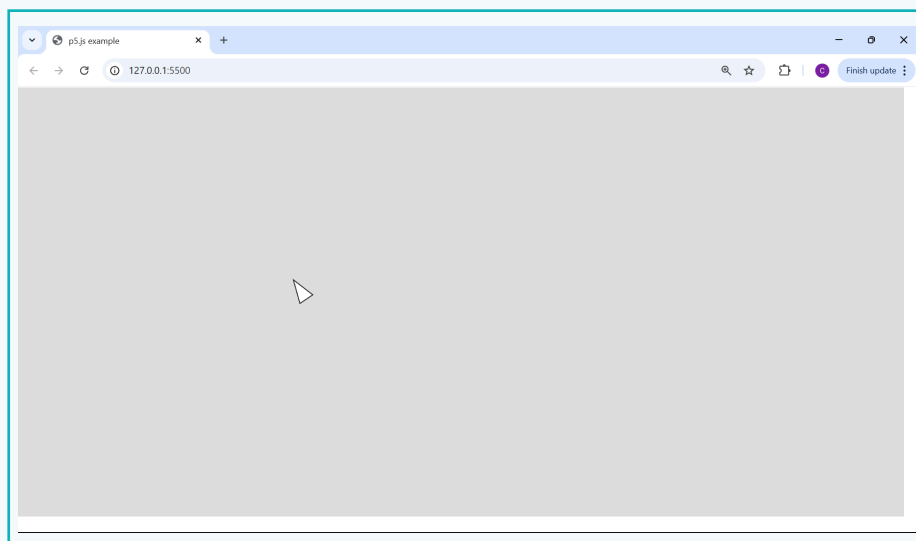
Activity

- 3 Open **VS Code** and the project folder in the editor.

The project files in the folder can be seen in the **Explorer** on the left-hand side.



- 4 Click on the 'Go live' option in the bottom bar or press 'Alt+L+O' to run the code. Check the output in the browser.



A vehicle object randomly moving on the grey-coloured canvas can be observed all over the browser window.

Activity

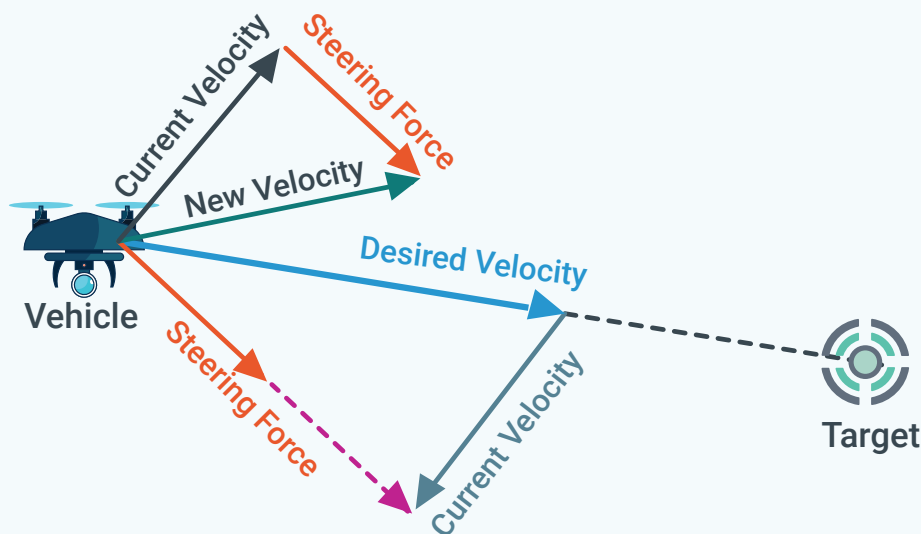
Note:

A class for creating an autonomous agent like the Mover class typically includes functions like *update()*, *applyForce()*, and *render()*, each handling specific tasks related to motion and display. To add a new **steering behavior**, we simply create a new function that calculates the steering **force vector** and pass it to the *applyForce()* function. This modular approach ensures that existing functions remain unchanged, maintaining code flexibility and reusability.

We can observe that, Mover class contains *randomWalk()* method and the vehicle moves randomly when code is executed. Now, we can start implementing various steering behaviors by creating new methods within the Mover class and applying them to the vehicle object.

The Steering Force

A steering force helps an autonomous agent adjust its movement intelligently. For example, imagine a drone attempting to land on a moving platform. Instead of being directly pulled toward it like gravity, the drone calculates how to steer based on its current speed, direction, and the platform's position.



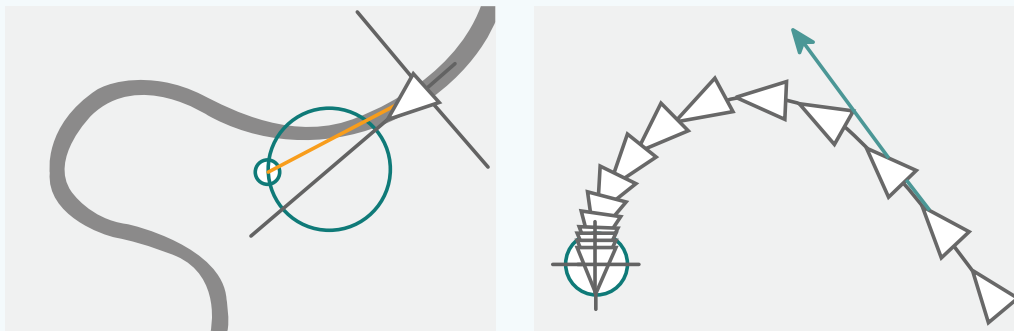
The drone determines how it **wants** to move (a vector toward the platform), compares it with how it is **currently** moving (velocity), and applies a force to adjust its path. This follows Reynolds's formula:

Activity

Now we can directly start implementing Steering Behaviors algorithms for the vehicle. We can implement **Steering Algorithms** such as seek, flee, arrive, etc., that can be used to control the vehicle's movement and make it autonomous to appear intelligent.

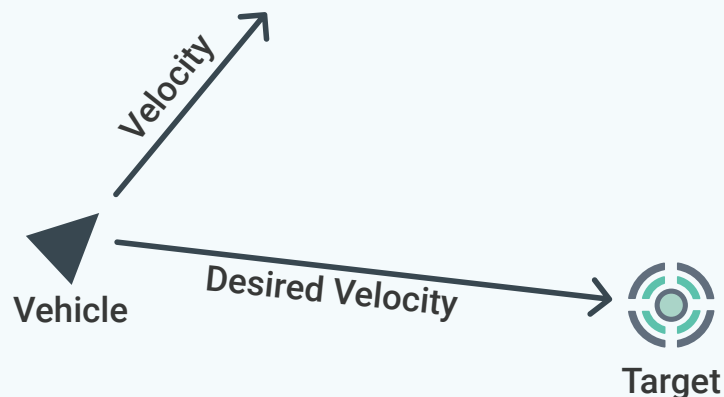
All these algorithms work together to influence the vehicle's acceleration or velocity. The implementation details vary based on the application or simulation in the p5.js framework.

Steering Force = Desired Velocity – Current Velocity



Part 2 - Seek Behavior

In 'seek' steering behavior, the vehicle detects a target and steers towards its position in space. This behavior adjusts the vehicle to align its velocity towards the target.



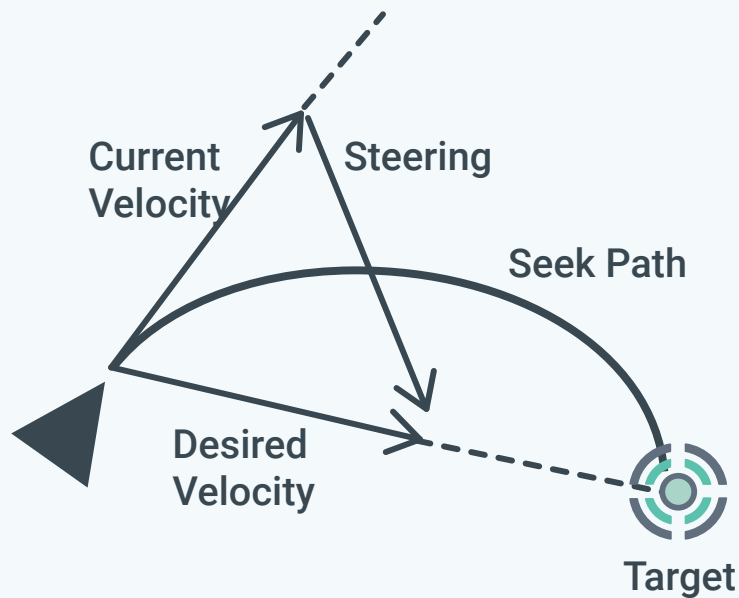
To seek a target, the vehicle needs to find a vector directing towards the target. We can call it the 'desired velocity' vector, which points in the direction from the vehicle to the target. It can be found by subtracting the target from the current position vector and updating its magnitude.

desired velocity = normalize (target - position) * max speed

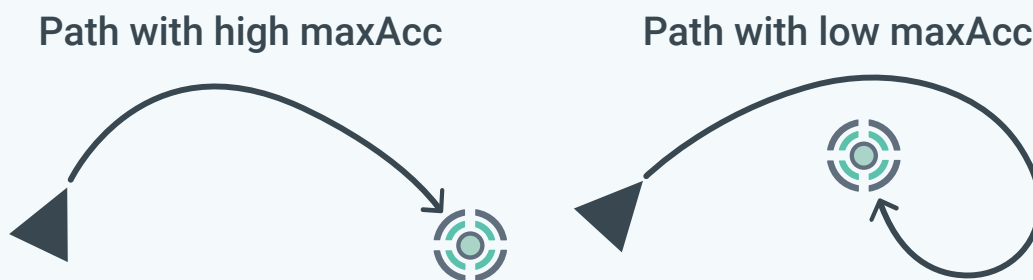
Activity

However, the vehicle already has a velocity. To shift from the current velocity to the desired velocity, we need to subtract the current velocity from the desired velocity to get the acceleration due to steering force.

$$\text{steering} = \text{desired velocity} - \text{current velocity}$$



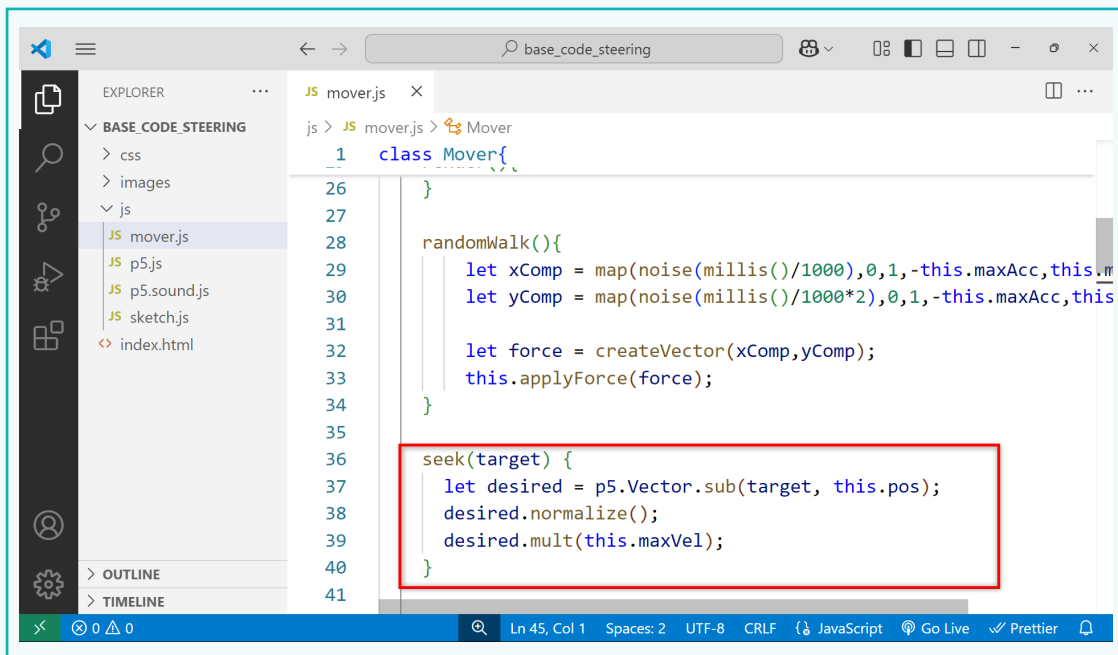
If the vehicle continues to seek, it may pass through the target after reaching it, as it doesn't slow down instantly. Also, the path of the motion depends on the limit of the steering force acceleration, which in our case is 'maxAcc'.



Activity

Let us add code for the same. We can create a new method named `seek` in the `Mover` class.

- 1 Go to the 'mover.js' file and create a new method named 'seek' with the input parameter `target` as `seek(target)`.
- 2 Subtract the current position vector from the 'target' vector and assign its output to a new vector variable 'desired' representing the desired velocity.
- 3 Get a unit vector from the desired vector using the `normalize()` method.
- 4 Set the magnitude of desired vector to 'maxVel' using `mult()` method.



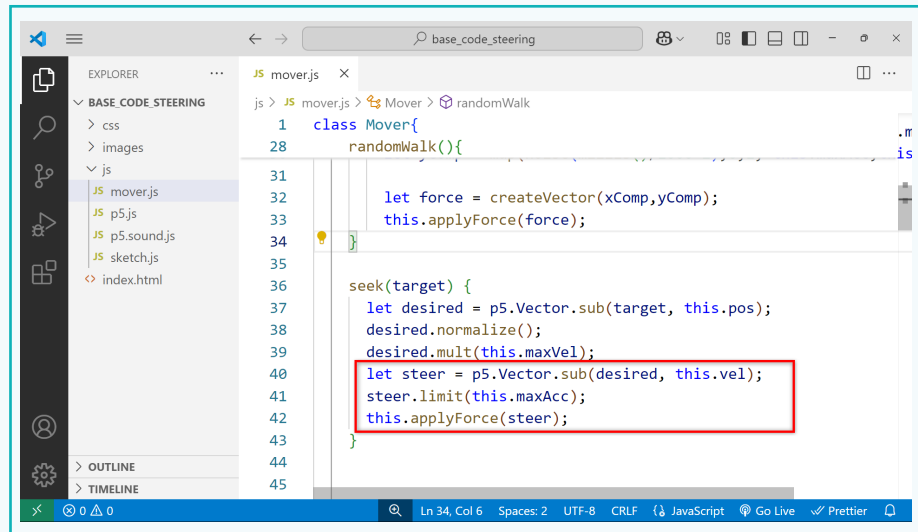
```
1 class Mover{
26
27
28   randomWalk(){
29     let xComp = map(noise(millis())/1000),0,1,-this.maxAcc,this.fr
30     let yComp = map(noise(millis())/1000*2),0,1,-this.maxAcc,this
31
32     let force = createVector(xComp,yComp);
33     this.applyForce(force);
34   }
35
36   seek(target) {
37     let desired = p5.Vector.sub(target, this.pos);
38     desired.normalize();
39     desired.mult(this.maxVel);
40   }
41 }
```

Note:

In JavaScript, 'this' refers to the current instance of the class. We must use 'this.' keyword when accessing class property variables or methods inside the class.

Activity

- 5 Subtract the current velocity from the desired velocity vector and assign the output to a new vector variable 'steer' representing steering force acceleration.
- 6 Limit the force magnitude to 'maxACC' using the limit method.
- 7 Use `applyForce()` method with the 'steer' vector as input.

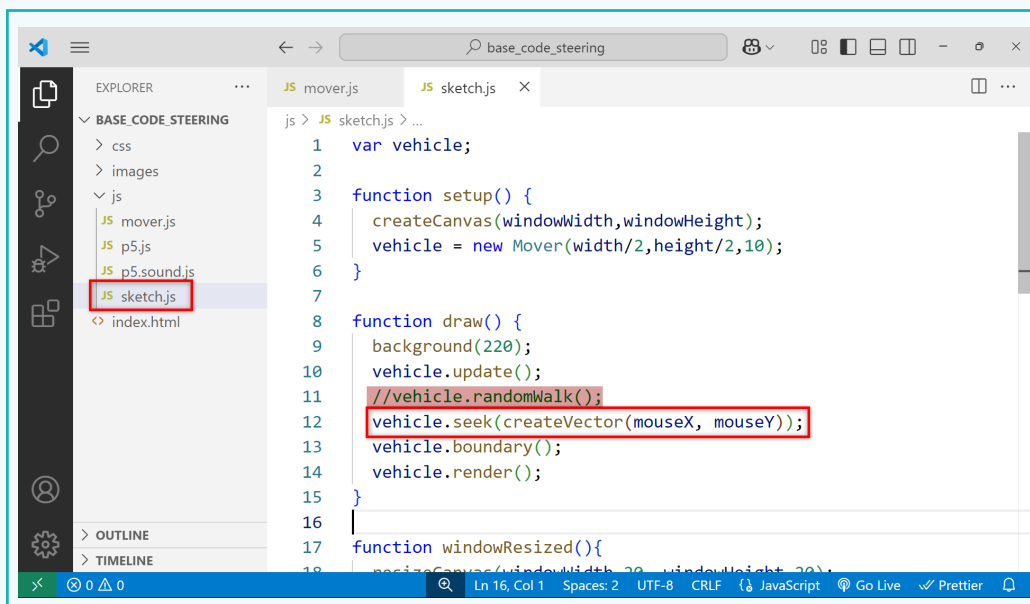


```
1 class Mover{
28   randomWalk(){
31
32     let force = createVector(xComp,yComp);
33     this.applyForce(force);
34   }
35
36   seek(target) {
37     let desired = p5.Vector.sub(target, this.pos);
38     desired.normalize();
39     desired.mult(this.maxVel);
40     let steer = p5.Vector.sub(desired, this.vel);
41     steer.limit(this.maxAcc);
42     this.applyForce(steer);
43   }
44
45 }
```

Try: Change the values of `maxVel` and `maxAcc` to check changes in the sharpness of seeking.

In `sketch.js`, the vehicle object has already been created, and all other methods like `render()`, `update()`, and `boundary()` have been implemented.

- 8 Go to the 'sketch.js' file and comment out the code `vehicle.randomWalk();` in `draw()` function to implement the seek behavior.
- 9 Add the `vehicle.seek()` method with a new vector as input, using `createVector(mouseX, mouseY)` for the cursor position.



```
1 var vehicle;
2
3 function setup() {
4   createCanvas(windowWidth,windowHeight);
5   vehicle = new Mover(width/2,height/2,10);
6 }
7
8 function draw() {
9   background(220);
10  vehicle.update();
11  //vehicle.randomWalk();
12  vehicle.seek(createVector(mouseX, mouseY));
13  vehicle.boundary();
14  vehicle.render();
15 }
16
17 function windowResized(){
18   createCanvas(windowWidth, windowHeight);
19 }
```

Activity

Check the Output



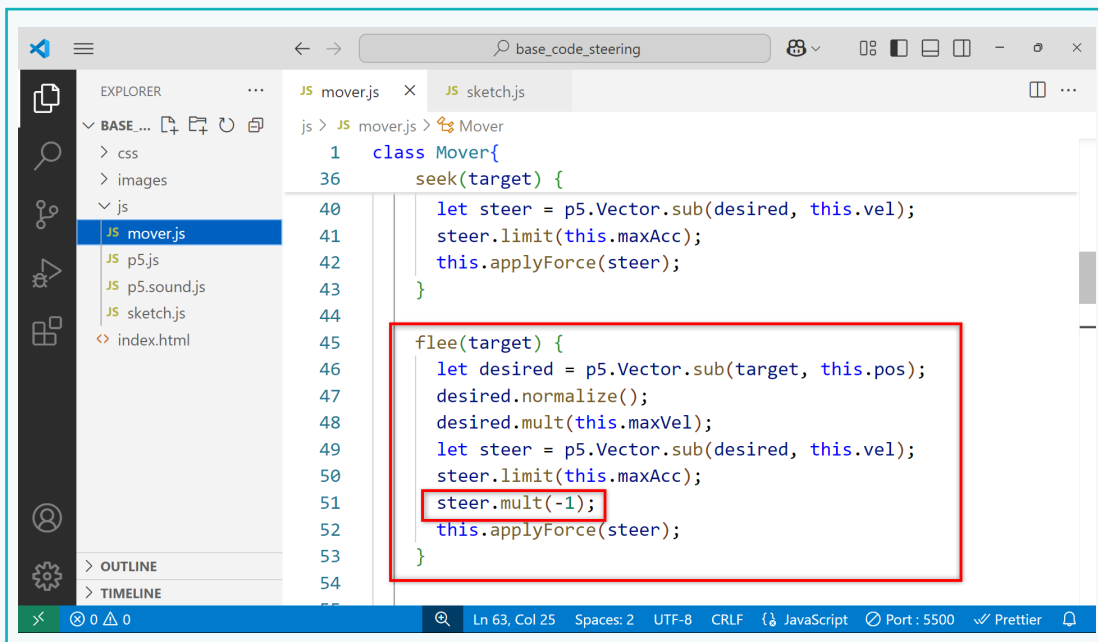
From the output, it can be seen that the vehicle seeks the mouse cursor wherever it moves. Additionally, once it reaches the cursor, it overshoots the position due to acceleration and continues moving around it.

Try changing the values of *maxVel* and *maxAcc* to vary the seek intensity.

Part 3 - Flee Behavior

We have achieved seek steering behavior. Flee steering behavior is exactly the opposite of seek. It can be achieved by negating the force vector achieved in *seek()* method.

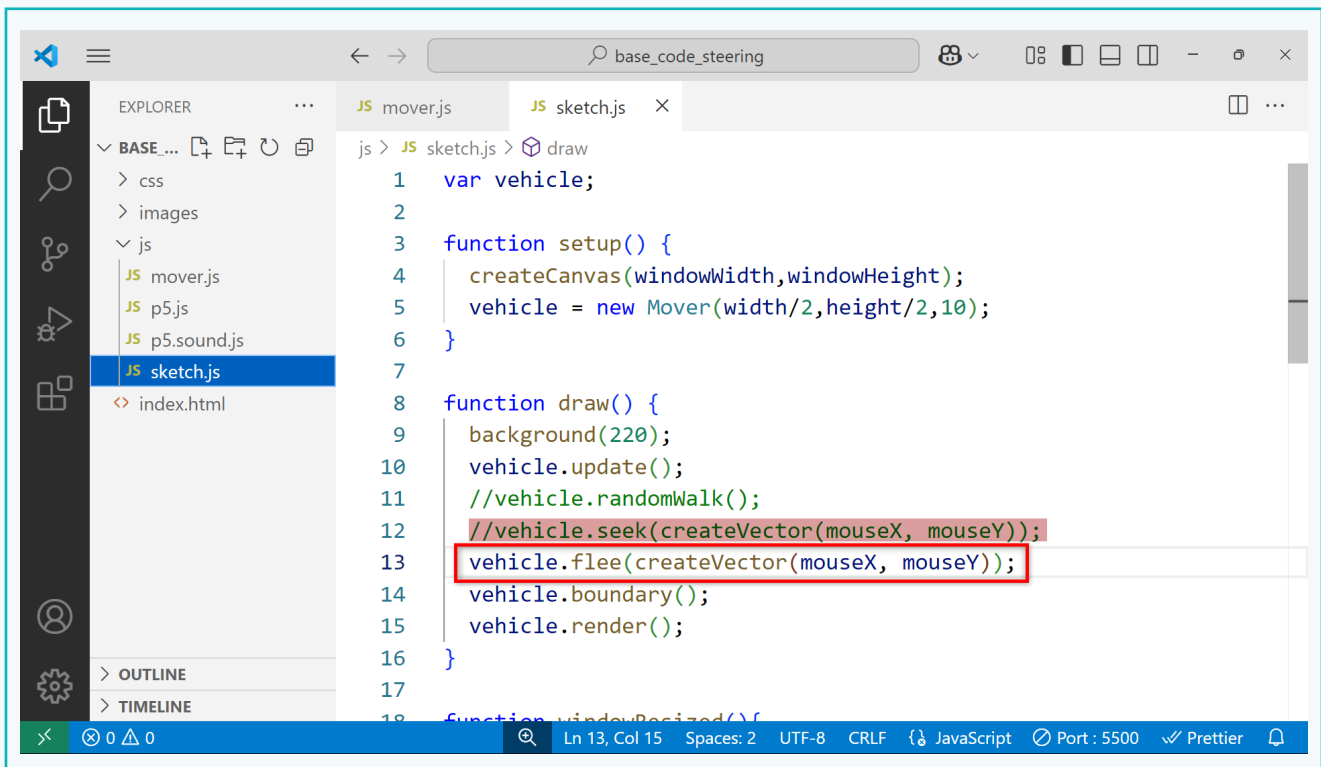
- 1 In the 'Mover' class, create a new method *flee(target)* with 'target' as a method parameter.
- 2 Copy the code from the *seek* method, as most of the code will be same for flee steering behavior.
- 3 Add *steer.mult(-1);* before the *applyForce()* is called.



```
1 class Mover{
36   seek(target) {
40     let steer = p5.Vector.sub(desired, this.vel);
41     steer.limit(this.maxAcc);
42     this.applyForce(steer);
43   }
44
45   flee(target) {
46     let desired = p5.Vector.sub(target, this.pos);
47     desired.normalize();
48     desired.mult(this.maxVel);
49     let steer = p5.Vector.sub(desired, this.vel);
50     steer.limit(this.maxAcc);
51     steer.mult(-1);
52     this.applyForce(steer);
53   }
54 }
```

Activity

- 4 Go to 'sketch.js' and comment out the call for the *seek* method, to implement the flee behavior.
- 5 Instead, call the *flee* method for the *vehicle* object, passing a vector with mouse cursor position variables *mouseX* and *mouseY* as input.



The screenshot shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project structure with files like `css`, `images`, `js`, `mover.js`, `p5.js`, `p5.sound.js`, `sketch.js`, and `index.html`. The code editor shows the following code:

```
js > JS sketch.js > draw
1 var vehicle;
2
3 function setup() {
4   createCanvas(windowWidth,windowHeight);
5   vehicle = new Mover(width/2,height/2,10);
6 }
7
8 function draw() {
9   background(220);
10  vehicle.update();
11  //vehicle.randomWalk();
12  //vehicle.seek(createVector(mouseX, mouseY));
13  vehicle.flee(createVector(mouseX, mouseY));
14  vehicle.boundary();
15  vehicle.render();
16 }
17
18 function windowResized() {
```

The code is in a file named `sketch.js`. The `draw` function contains the logic for the vehicle's movement. The `vehicle.flee(createVector(mouseX, mouseY));` line is highlighted with a red box, indicating the current focus of the activity.

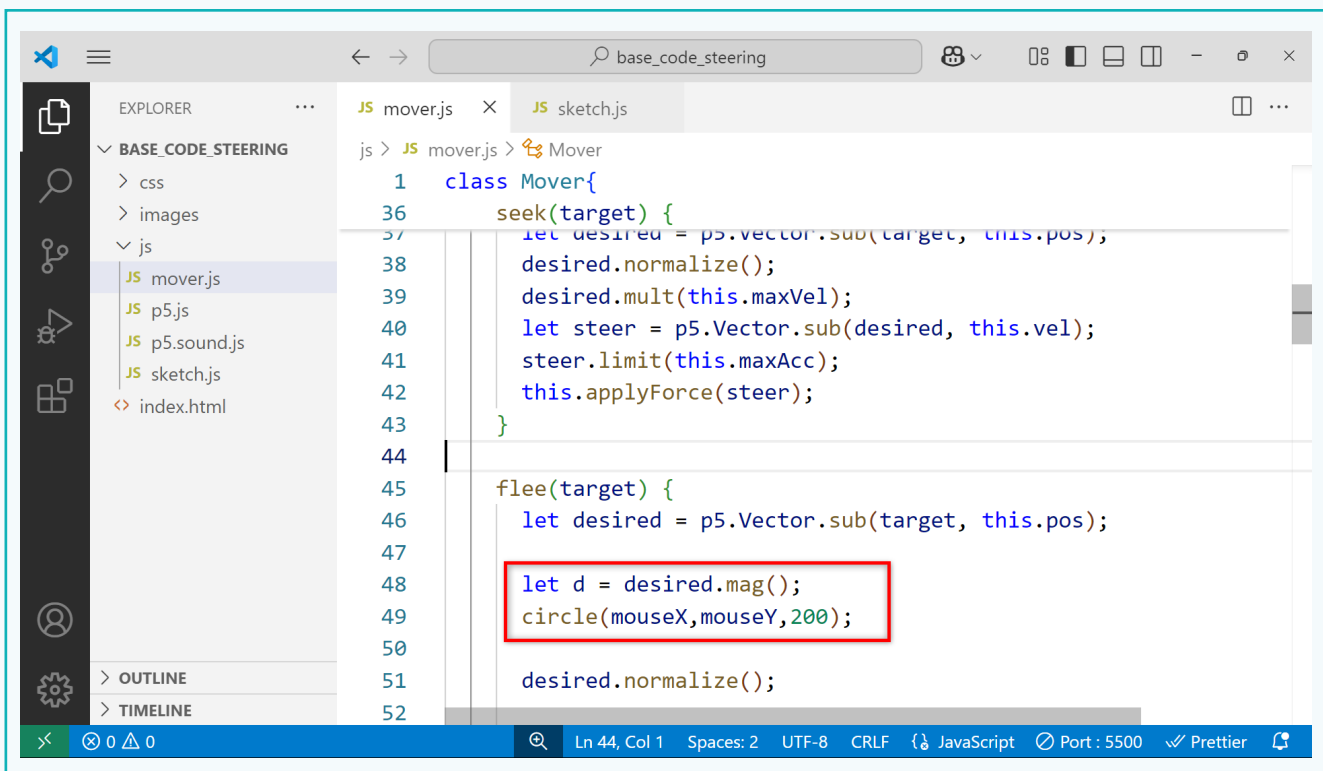
Now if we observe the output, the vehicle keeps moving out of the screen. This is because it tries to flee from the cursor as soon as it enters the screen.

To better observe the effects of the flee method, let us update the code so that the vehicle flees only when it enters a certain vicinity of the mouse cursor. Additionally, mark this area with a circle.

Activity

To check if the vehicle is in the vicinity of the target i.e., mouse cursor.

- 6 Calculate the distance between the vehicle and the cursor by getting the magnitude of the desired vector using the *mag()* method.
- 7 Create a variable 'd' and assign the magnitude value to it.
- 8 Also, draw a circle of 200 diameters at the position of the target, i.e., the mouse cursor, to showcase the area of vicinity.

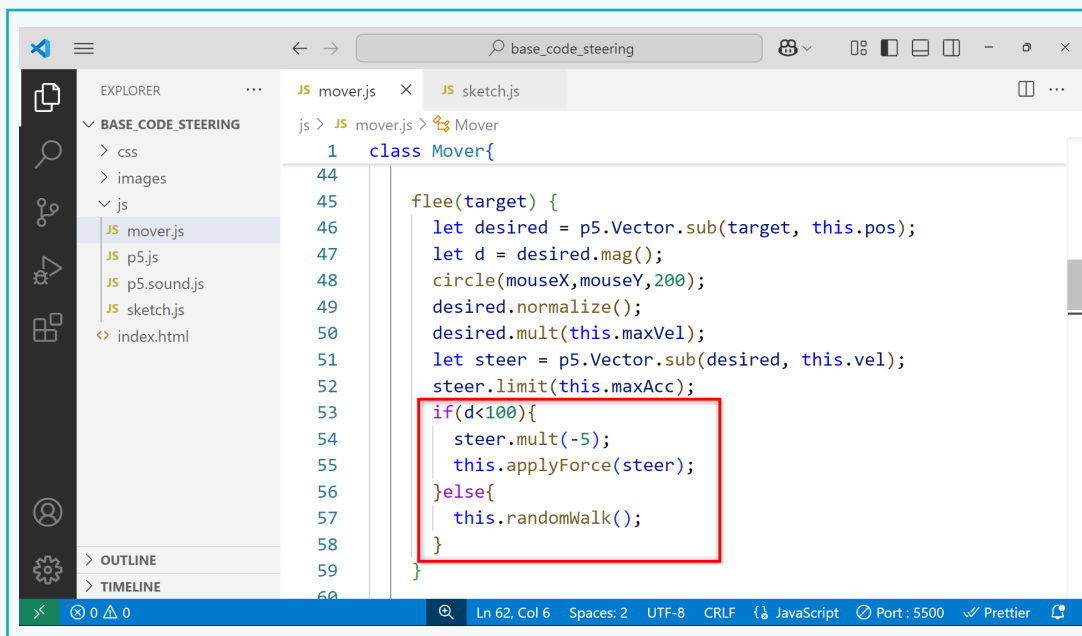


```
1 class Mover{
36   seek(target) {
37     let desired = p5.Vector.sub(target, this.pos);
38     desired.normalize();
39     desired.mult(this.maxVel);
40     let steer = p5.Vector.sub(desired, this.vel);
41     steer.limit(this.maxAcc);
42     this.applyForce(steer);
43   }
44
45   flee(target) {
46     let desired = p5.Vector.sub(target, this.pos);
47
48     let d = desired.mag();
49     circle(mouseX, mouseY, 200);
50
51     desired.normalize();
52   }
```

Now, let's add a condition to check if the vehicle is near the target. It will flee only when within range; otherwise, it will move in random directions.

Activity

- 9 Add a condition *if* ($d < 100$), to check if the vehicle enters the 200-diameter circle around the mouse cursor.
- 10 Move *steer.mult(-1)* and *this.applyforce(steer)* code lines inside the condition, so that the flee steering force activates when the vehicle is within 100 pixels of the target.
- 11 Multiply -1 by 5 in *mult()* command such that the flee intensity is higher.
- 12 Add an *else* part to the condition and call the *randomWalk()* method using the *this* keyword, so that the vehicle moves randomly when it is outside the defined vicinity.



```
1 class Mover{
44
45   flee(target) {
46     let desired = p5.Vector.sub(target, this.pos);
47     let d = desired.mag();
48     circle(mouseX, mouseY, 200);
49     desired.normalize();
50     desired.mult(this.maxVel);
51     let steer = p5.Vector.sub(desired, this.vel);
52     steer.limit(this.maxAcc);
53     if(d < 100){
54       steer.mult(-5);
55       this.applyForce(steer);
56     }else{
57       this.randomWalk();
58     }
59   }
60 }
```

Check the Output

The output shows that the vehicle will flee only when it enters the circle around the mouse cursor.

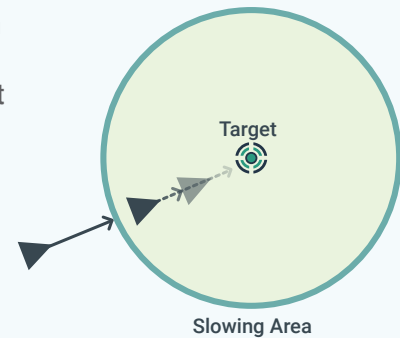


Activity

Part 4 - Arrive Behavior

In Seek Behavior, a vehicle moves directly toward a target at maximum speed and applies a steering force to align its movement. The vehicle tries to go as fast as possible towards the target. However, this forces it to overshoot and then turn around to go back.

Arrive Behavior improves upon Seek by gradually reducing the speed as the vehicle approaches the target, creating a smooth and realistic motion. For example, a car approaching a red light doesn't stop abruptly but slows down gradually until it comes to a halt smoothly. Arrive behavior replicates such an effect in simulations and games.



We can implement this by modifying seek behavior such that if the distance from the target is less than a certain distance r , the desired speed ranges from maximum velocity magnitude to 0, mapped according to that distance.

- 1 Copy and paste the `seek(target)` method inside the 'Mover' class and rename it to `arrive()`.
- 2 After the desired vector is calculated, create a variable 'd' and assign it the magnitude of the 'desired' vector calculated using 'mag()' method. Just like we calculated this in Flee, this will provide the distance between the vehicle and the target.

Activity

- 3 To change the velocity based on the distance between vehicle and target, add an *if* condition to check *if(d<200)*.
- 4 Inside the conditional block, create a variable *slowSpeed*. Use the *map()* command to calculate the speed so that as distance between the vehicle and the target varies between **0** to **200**, the speed varies from *0* to *maxVel*. Assign it to *slowSpeed* variable.

- 5 Multiply the *desired* vector by the magnitude provided by *slowSpeed* variable.
- 6 Add 'else' section and move *desired.mult(this.maxVel)*; inside it to maintain normal seek behavior when the distance is greater than **200**.

```

class Mover {
  arrive(target) {
    let desired = p5.Vector.sub(target, this.pos);
    let d = desired.mag();
    desired.normalize();

    if(d<200){
      let slowSpeed = map(d, 0, 200, 0, this.maxVel);
      desired.mult(slowSpeed);
    }else{
      desired.mult(this.maxVel);
    }

    let steer = p5.Vector.sub(desired, this.vel);
    steer.limit(this.maxAcc);
    this.applyForce(steer);
  }
}

```

- 7 Implement the *arrive()* method for *vehicle* object in 'sketch.js' while commenting code for earlier behavior.
- 8 Execute the code to see *arrive()* behavior implemented.

```

function setup() {
  createCanvas(windowWidth, windowHeight);
  vehicle = new Mover(width/2, height/2, 10);
}

function draw() {
  background(220);
  vehicle.update();
  //vehicle.randomWalk();
  //vehicle.seek(createVector(mouseX, mouseY));
  //vehicle.flee(createVector(mouseX, mouseY));
  vehicle.arrive(createVector(mouseX, mouseY));
  vehicle.boundary();
  vehicle.render();
}

```

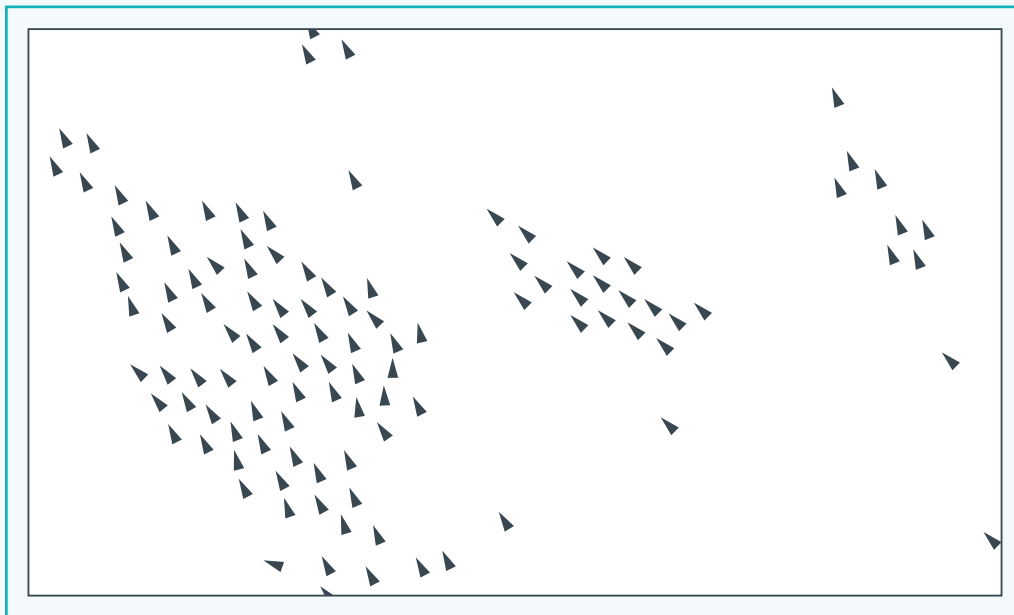

Activity

Check the Output

The arrive behavior is a great demonstration of an autonomous agent's i.e., vehicle's perception of the environment along with its own state.



There are many other interesting behaviors, such as wandering, pursuing, evading, path following, and more, that you can explore online and try to implement. Combining different steering behaviors can lead to many real-life phenomena and also replicate scenarios like flocking.



Understanding how objects move realistically in a digital environment is essential for creating lifelike simulations and interactive experiences. By applying OOP principles, vectors, laws of motion, and steering behaviors, autonomous agents can navigate their surroundings intelligently.

In this lesson, we implemented **steering behavior** for the vehicle, such as seeking and arriving, which enable vehicles to adjust their movement dynamically rather than following predefined paths.

Note:

Lessons for autonomous agents and steering behaviors use **Craig Reynolds** paper on steering behaviors and **Daniel Shiffman's** 'Nature of Code' as base reference.

Exercise

 **Tick Mark the Correct Answer.**

- 1** What does the steering force equation help calculate?
 - a. The mass of an object
 - b. The force needed to adjust a vehicle's movement
 - c. The target's exact position
 - d. The colour of the vehicle

- 2** Which command is used to create a vector?
 - a. `createAgent()`
 - b. `applyForce()`
 - c. `createVector()`
 - d. `moveObject()`

- 3** What is the main purpose of steering behaviors in autonomous agents?
 - a. To create random movement in virtual environments
 - b. To allow agents to move intelligently and respond dynamically
 - c. To make agents move in straight lines only
 - d. To make the game look more colourful

- 4** Which of the following is NOT a common steering behavior?
 - a. Seek
 - b. Flee
 - c. Random Walk
 - d. Teleport

- 5** What does the flee behavior do?
 - a. Moves toward a target
 - b. Moves away from a target
 - c. Stops when close to a target
 - d. Moves in a straight line

Exercise

- 6 In seek behavior, what is subtracted to determine the desired velocity?
- a. Target velocity – Acceleration
 - b. Current velocity – Desired velocity
 - c. Target position – Current position
 - d. Steering force – Acceleration
- 7 What happens in arrive behavior?
- a. The vehicle moves toward a target at constant speed
 - b. The vehicle slows down as it reaches a target
 - c. The vehicle moves away from the target
 - d. The vehicle teleports to the target

State if the Statements are True or False.

- 8 The steering force is calculated as the difference between desired velocity and current velocity.

▲ _____

- 9 Desired velocity represents the ideal speed and direction a vehicle should move in.

▲ _____

Tech Flash

Autonomous Agents	: Virtual objects that possess a degree of autonomy in generating their own movements and actions.
Steering Behaviors	: Algorithms that help virtual agents move dynamically, avoiding obstacles, chasing targets, or patrolling areas.
Seek Behavior	: A steering technique where a vehicle moves toward a target by adjusting its velocity.
Flee Behavior	: A movement strategy where a vehicle moves away from a target, often used for evasion.
Arrive Behavior	: A steering behavior where a vehicle slows down as it reaches a target, preventing overshooting.
Steering Force	: The force applied to change vehicle's velocity, calculated as (desired velocity - current velocity).
Desired Velocity	: The velocity required to move towards or away from a target in steering behaviors.